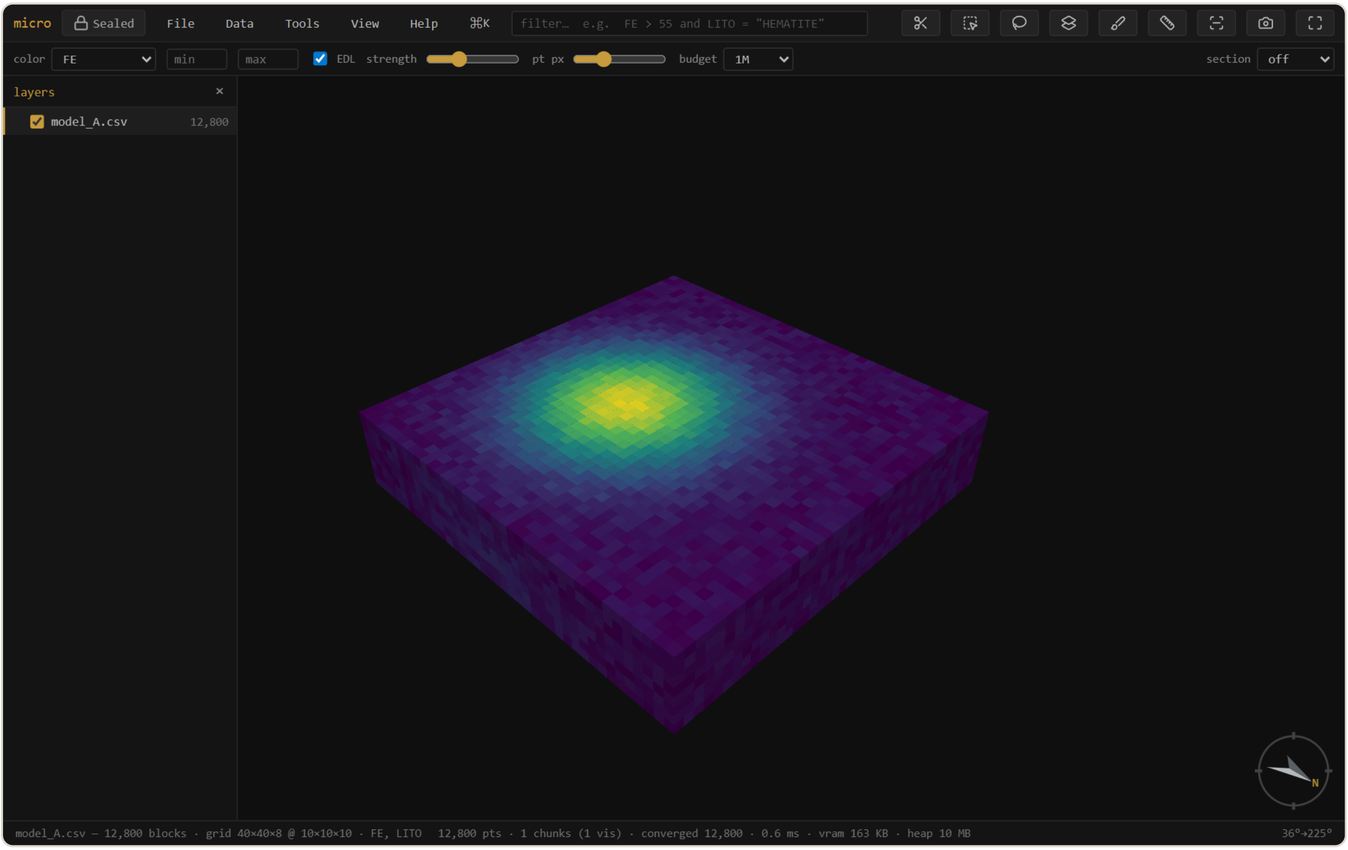
micro

A streaming, offline workbench for massive point clouds and block models.

🔒 **Sealed** — no network, nothing leaves your machine · single self-contained HTML file · download micro.html · open the app ↗

User documentation · Auditable / Geoscientific Chaos Union · gentropic.org/micro

micro opens a point cloud, block model, grid, or set of drillholes — however large — in about a second, with no import step, and lets you view, query, join, reconcile, validate, and produce report-ready figures from it. Every derived result carries a provenance trail, and nothing ever touches the network.



A 12,800-block iron model coloured by grade — the high-grade shoot in yellow. Loaded, inferred, and rendered from a plain CSV with no import step.

Contents

1 What micro is

micro is a single self-contained HTML file — a **viewer that grew into a workbench**. It streams massive spatial-element files (never holding them fully in memory), so the first look is near-instant at any size and the picture densifies while you orbit.

It reads:

- **Point clouds** — LAS, PLY, XYZ / PTS / DAT dumps
- **Block models** — CSV / TXT, Datamine `.dm`, and **Parquet**
- **Grids** — GeoTIFF, Surfer `.grd`, ESRI ASCII `.asc`
- **Meshes** — `.msh` (Leapfrog), OBJ, PLY-with-faces, LFM
- **Drillholes** — collar + survey + interval tables (desurveyed on load); a set can carry **several interval tables** (assays, lithology, geotech) as sibling layers sharing one geometry, with hole traces on demand

Beyond viewing, it does the everyday resource-geology work: filter and section, domain by a solid, join and reconcile model versions, grade–tonnage and swath analysis, validation against drillholes, and figure export — all against the source file, all with a provenance trail, all offline.

> **Tip —** The fastest way to reach anything is the **command palette**: `Ctrl/⌘K` (or the `⌘K` button). It searches every action and shows only what applies to the layer you have selected.

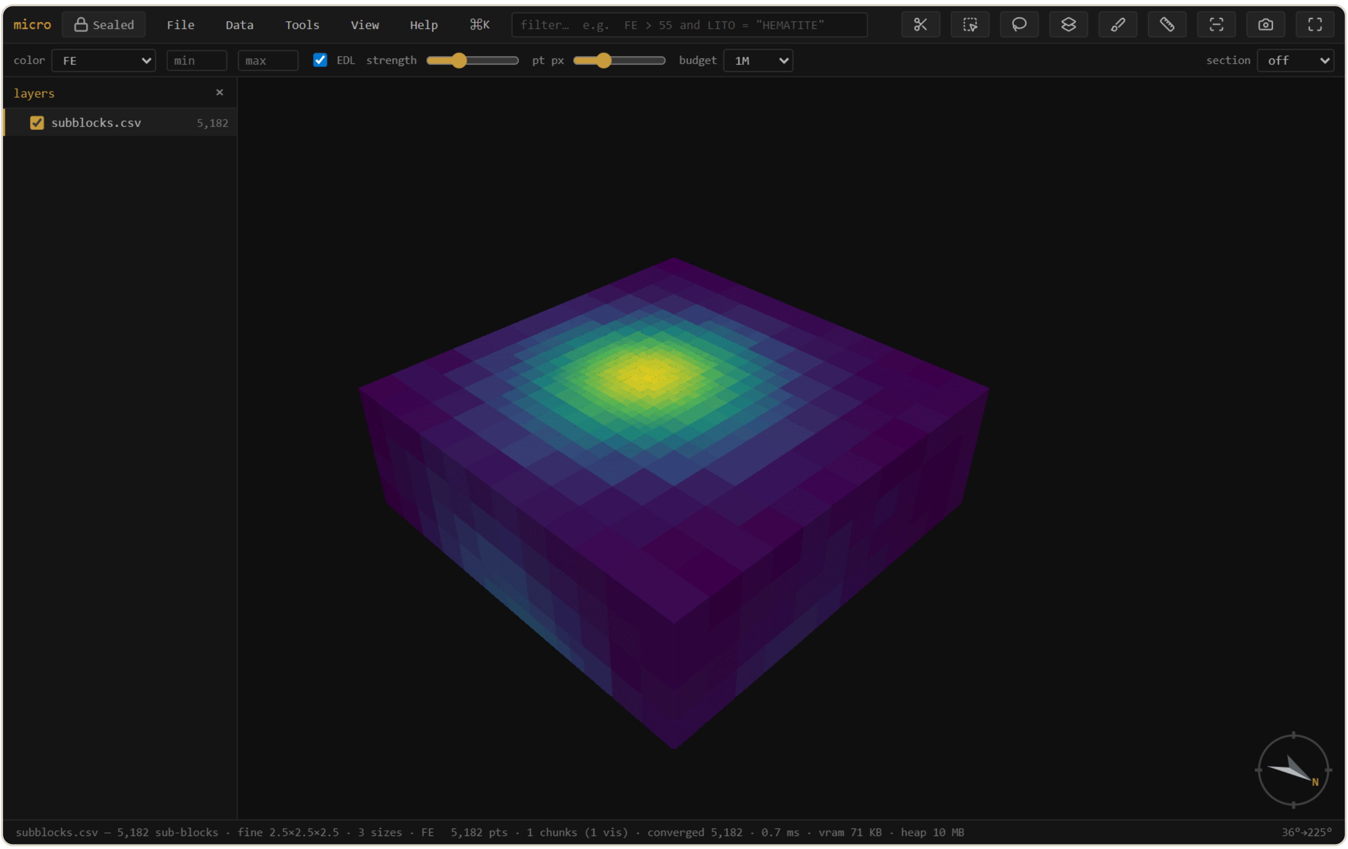
2 Getting started

Opening data

Drag a file (or several) anywhere onto the window, or use **File** → **Open…**. Several files at once become separate **layers**. The open dialog shows each file's detected role — an **Open as** ▾ on every row lets you override it (a block model as points, a PLY mesh as points, a delimited file as collar / survey / intervals), and drillhole trios assemble themselves from those roles. No data handy? **File** → **Sample bench scan** or **Sample block model** generate a dataset locally.

Sub-blocked models

micro reads **sub-blocked** (octree) block models — where cells are refined into smaller boxes near a boundary or a high-grade zone — and renders **each block at its true size**, straight from the file. CSV/Parquet models with per-block `dX/dY/dZ` columns and Datamine `.dm` models with per-record `XINC/YINC/ZINC` are detected automatically; the model streams, picks, filters, and colours like any other. Join and reconcile handle them by **volume** — see Join & reconcile.



A sub-blocked (octree) model — coarse parent blocks away from the shoot, refined into progressively smaller boxes through the high-grade core, each drawn at its own size.

> **Note —** Detection assumes an **axis-aligned** model with power-of-two (octree) refinement — the standard export. A **rotated** model currently opens as its centroids (points); interpreting a rotated model as boxes (enter or measure its azimuth) is on the roadmap.

The layers panel

`p` toggles the layers panel. Each opened dataset is a layer with its own colour, filter, and visibility; right-click a layer for its actions (zoom, info, attribute table, filter, analysis tools, remove). Layers can be grouped and reordered; `[` `]` step the active layer, `h` hides/shows it, `⇧H` **solos** it (press again to restore what was visible).

`F2` renames the active layer. **Rename layers…** (on a group, a multi-selection, or the palette) batch-renames: one-click **strip the common prefix/suffix**, or find→replace with optional **regex**, with a live before→after table — handy when a folder of `BM2024_kriging.csv` -style names comes in.

Getting around

Drag to orbit, right-drag (or shift-drag) to pan, wheel to zoom. `f` fits the data. `n s e w` look level that way, `d` is plan, `o` toggles orthographic (needed for a true scale bar). Click a block or point to **pick** it — the record panel shows its full source row, with a **fields** picker (search + all/none) to trim it to the columns you care about.

3 Viewing & querying

Colour

The **colour** dropdown drives the active layer: by elevation, a numeric **channel** (a grade), or a **category** (a rock type). Pick a ramp per layer (viridis · spectral · magma · turbo · greys) in **Properties** → **style**; the **min/max** boxes clip the scale so outliers stop eating the ramp. On a grade channel, double-click the histogram to add **breakpoints** — **classed** turns them into a solid-colour grade-shell legend.

Filter

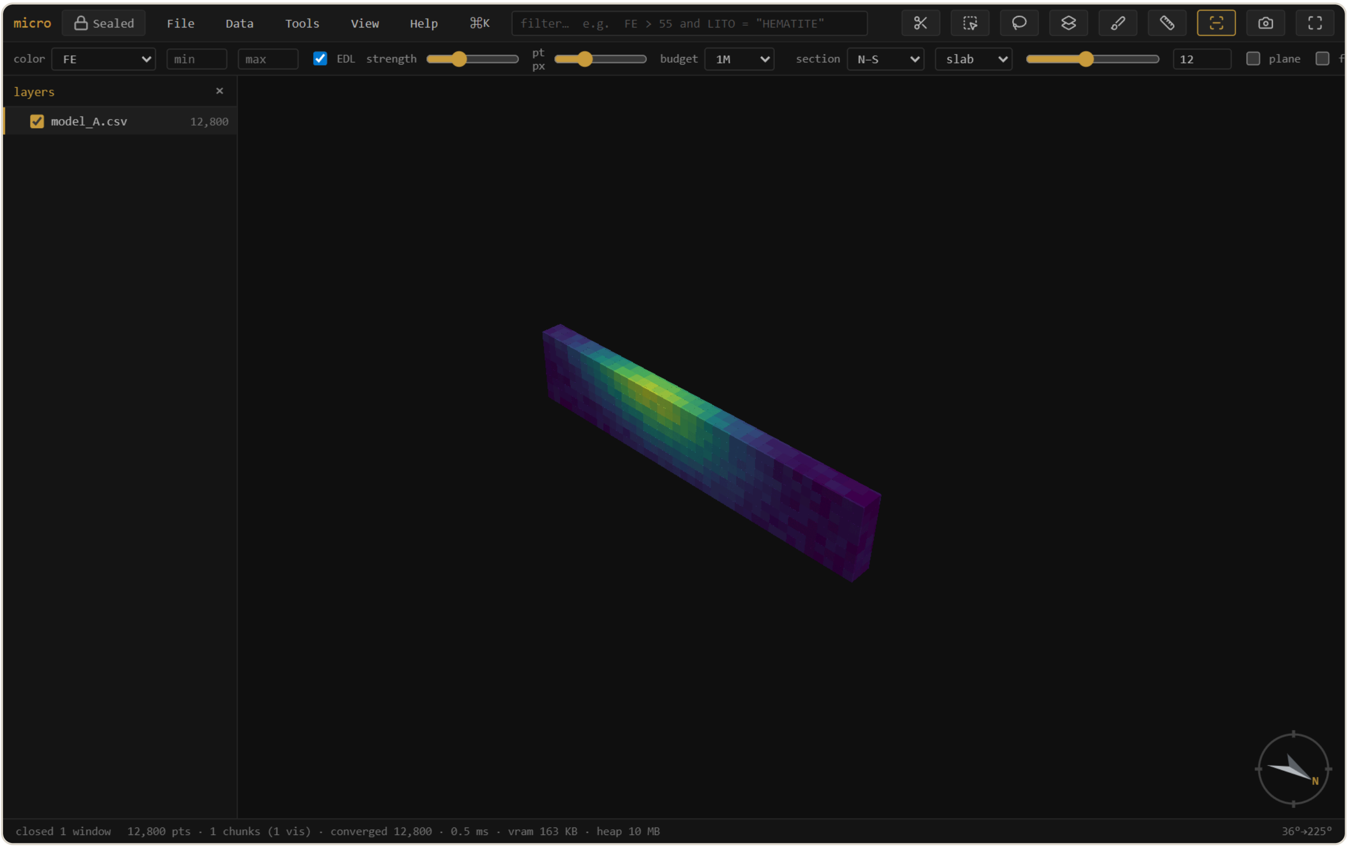
`/` focuses the filter box. It speaks a familiar expression language over every column: `FE > 55 and LITO = "HEMATITE"`, `between`, `in (…)`. Matching elements isolate; everything else drops out of the render, the table, and any analysis. `Enter` applies, `Esc` clears.

Calculated columns & the scan

Properties → **columns** defines ƒ **calculated columns** (`FE + 0.4 * MN`) that the filter, scan, table, and record panel all see, and **scan columns** streams the whole table once to fill n · mean · std · quantiles · histogram per column.

Sections & measure

A section is a live cut, and on block models it is a **true** cut: each block is clipped analytically against the slab faces, so the cut paints a continuous wall — with the block's colour — instead of a ragged centroid cull, and the wall itself is pickable. Meshes stay hollow but draw *over* the cut, so a wireframe's trace lines read on the wall.



A thin N–S slab. The cut face is a continuous coloured wall — blocks are clipped against the plane, not dropped by centroid — and clicking the wall picks the block behind it.

Pick a preset (plan · N–S · E–W), press `k` and drag a line (⇧ snaps 15°, ⇧⌥ 5°), or type a plane's dip-direction + dip. `x` toggles it, `l` faces it, `⇧L` faces its back. `,` `.` step one slab at a time, and **Ctrl+wheel** scrubs the position continuously. **Follow** (next to the section controls) makes the camera ride along as you scrub; **2D** locks the view flat onto the section — drags pan instead of orbit — for section-by-section interpretation. `m` then two clicks **measures** 3D · plan · Δz between records.

Block edges

`⇧B` (or View → block edges) outlines every block — the classic wireframe-over-solid look for checking sub-blocking or lattice alignment. Each layer can override the global toggle from its context menu (always / never / follow).

The attribute table

`t` opens the layer's rows as a draggable, layer-pinned window — windowed reads, so a 1.8M-row model scrolls cold. **matches** shows only the filtered rows; click a row to pick it in the scene.

4 Selection & domaining

Select with `r` (rectangle) or `v` (lasso): what you *see* gets selected (occlusion-honest, all visible layers), tinted gold — **shift** adds, **alt** subtracts, `Esc` clears. The **select-through** toolbar toggle flips to the extruded tube: everything whose position falls inside the shape, occluded included.

Select by solid / surface (Tools menu, or right-click a mesh) classifies records against a closed mesh by winding number: **select** inside/outside, write a **flag column**, or a **fraction-inside** 0–1 column. Open surfaces (topo, weathering horizons) classify **above / below / between**. **Assign domains** runs the whole coded-model loop — pick the domain solids and a column name, and every block goes to the solid holding its largest fraction.

Paint a column (`b`) is interactive domaining by hand: a category column you fill by brushing over the scene. Painted values are real columns the filter, table, and export all see.

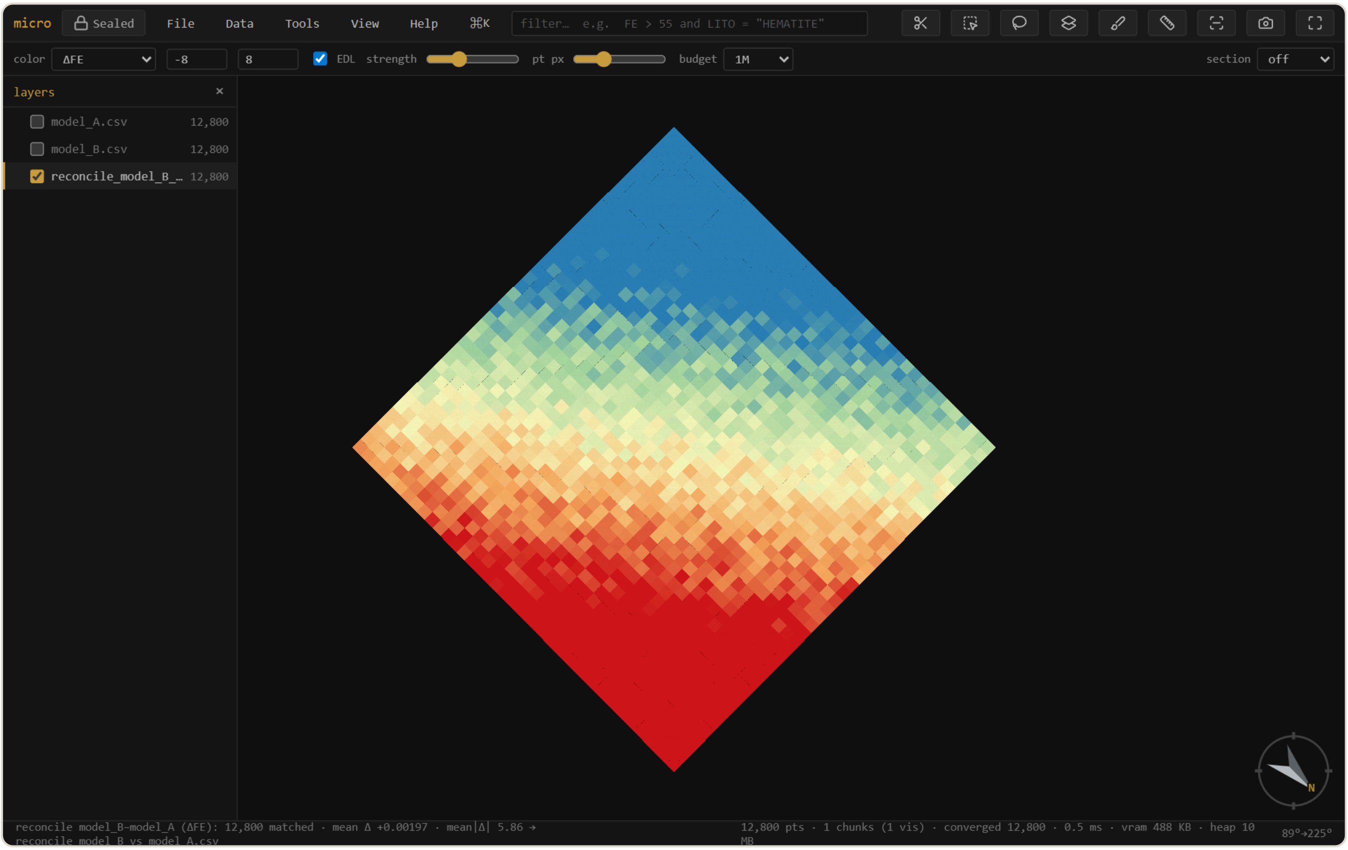
Selection → column… (right-click the layer, or the palette) materializes the current selection as an explicit **0/1 column** — every row gets a value, so it filters (`SEL = 1`), edits like any painted column, and rides every export. Selections themselves also persist in the project.

5 Join & reconcile

Two model versions? The **join tab** (Properties → **join**) compares them. A **spatial join** resamples a *compatible* model onto a target grid — a banner tells you whether the grids share a lattice (clean replicate / aggregate) or why they can't. A **key join** matches rows by a key column (e.g. a block ID, or a domain→density lookup).

Reconcile is the preset: it produces a coloured **Δ map** (this model − reference) on a common lattice, with a diverging ramp centred on zero — blue where the new estimate is lower, red where it's higher, neutral where they agree. The summary reads **matched · mean Δ · mean|Δ| · added · removed**.

Sub-blocked models reconcile by **volume**: each variable-size block is aggregated onto the reference (parent) grid weighted by the space it fills — a big parent and its small sub-blocks each contribute in proportion — so a sub-blocked estimate compares cleanly against a regular one at the parent resolution.



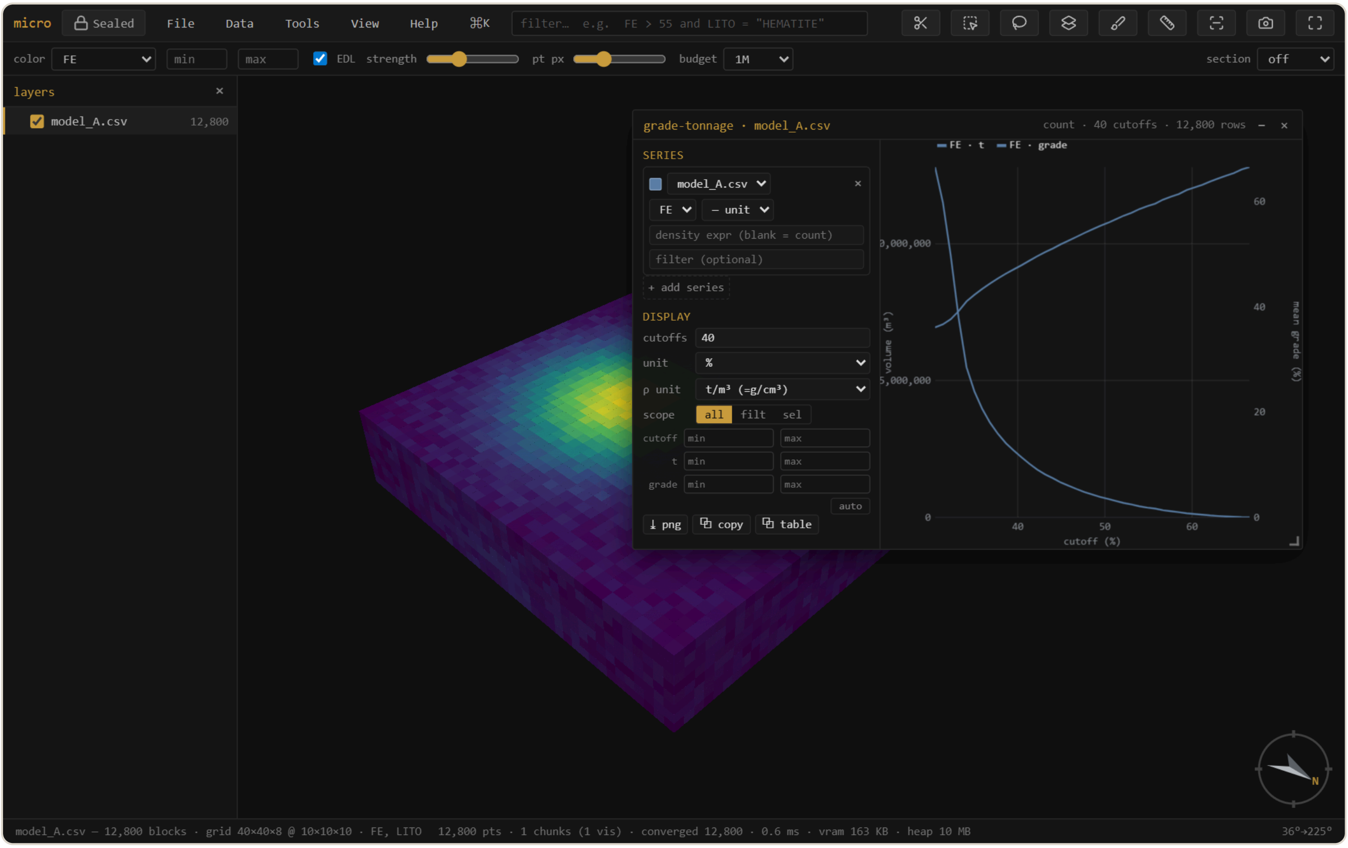
Reconciling two model versions. Global mean Δ is ≈ 0, but the Δ map exposes a clear **conditional bias** — the new estimate runs low in the north (blue) and high in the south (red). A global check would have missed it entirely.

> **Tip —** The Δ ramp auto-clips to the full range, which extreme blocks can widen. Tighten the **min/max** boxes (e.g. ±2× the mean|Δ|) to make the local pattern read, as above.

6 Grade–tonnage

Grade-tonnage… (right-click a model, or the palette) draws tonnage and mean grade above cutoff. It shares the swath's interface: a config rail of **series**, each one a **layer + grade column + declared unit + density expression + optional row filter** — configure, then **Run**. Because series pick their own layer, model-vs-model curves (this estimate against the last one) are one window; on a reconcile layer the reference/new pair is pre-seeded.

- **Density** is a free expression (`SG`, `2.7`, `SG * PARTIAL`) — tonnage = density × block volume, with a declared density unit converted to t/m^3 so tonnes are tonnes. Blank = count/volume.
- **Units are declarations**, not display toggles: declare what each series' grade *is* (%, g/t, ppm, oz/t) and a **plot unit** — series convert onto the shared axis honestly, so a %-model overlays a ppm-sample set without lying.
- One streamed scan per Run caches each series' cumulative curve, so **cutoff count, plot unit, and manual axis ranges are live afterwards** — no re-scan.
- ⤓ **png** saves the plot, ⧉ **copy** puts it on the clipboard, and ⧉ **table** copies the numbers as TSV — cutoff · tonnes · grade · contained metal per series, ready for a spreadsheet.



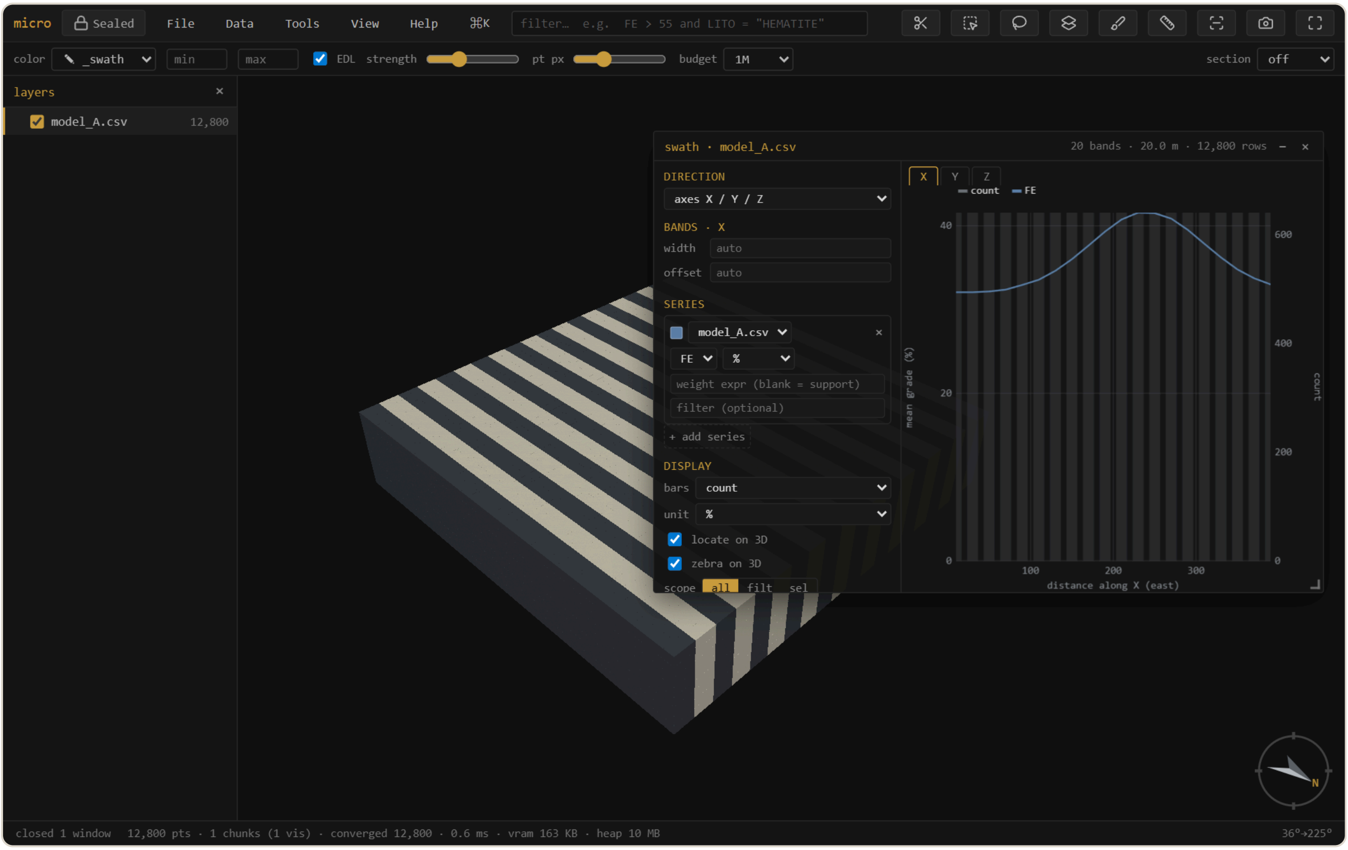
Grade–tonnage: tonnage (left axis) falls and mean grade (right axis) rises as the cutoff climbs — the recoverable-resource trade-off at a glance. The rail holds the series, units, density expression, scope, and manual ranges.

7 Swath / drift

Swath plot… shows mean grade per band along a direction — the classic drift check for local bias. The direction can be the grid **axes** X/Y/Z, an oriented trio from **dip-direction / dip / rake** (across-structure · along-strike · down-dip), or a single **trend / plunge**. Series work exactly like grade–tonnage's: layer + column + declared unit + a free **weight expression** (blank = block volume) + an optional per-series row filter — so kriging vs nearest-neighbour vs declustered samples overlay in one plot.

One **Run** streams everything once into fine bins; after that, **band width and offset re-bin instantly** — and they are **per direction**, so your E–W drift keeps its 25 m bands while the bench-by-bench Z profile keeps 10 m. Manual axis ranges, ⭳ png / ⧉ copy / ⧉ **table** (band · count · weight · per-series mean, TSV) match grade–tonnage.

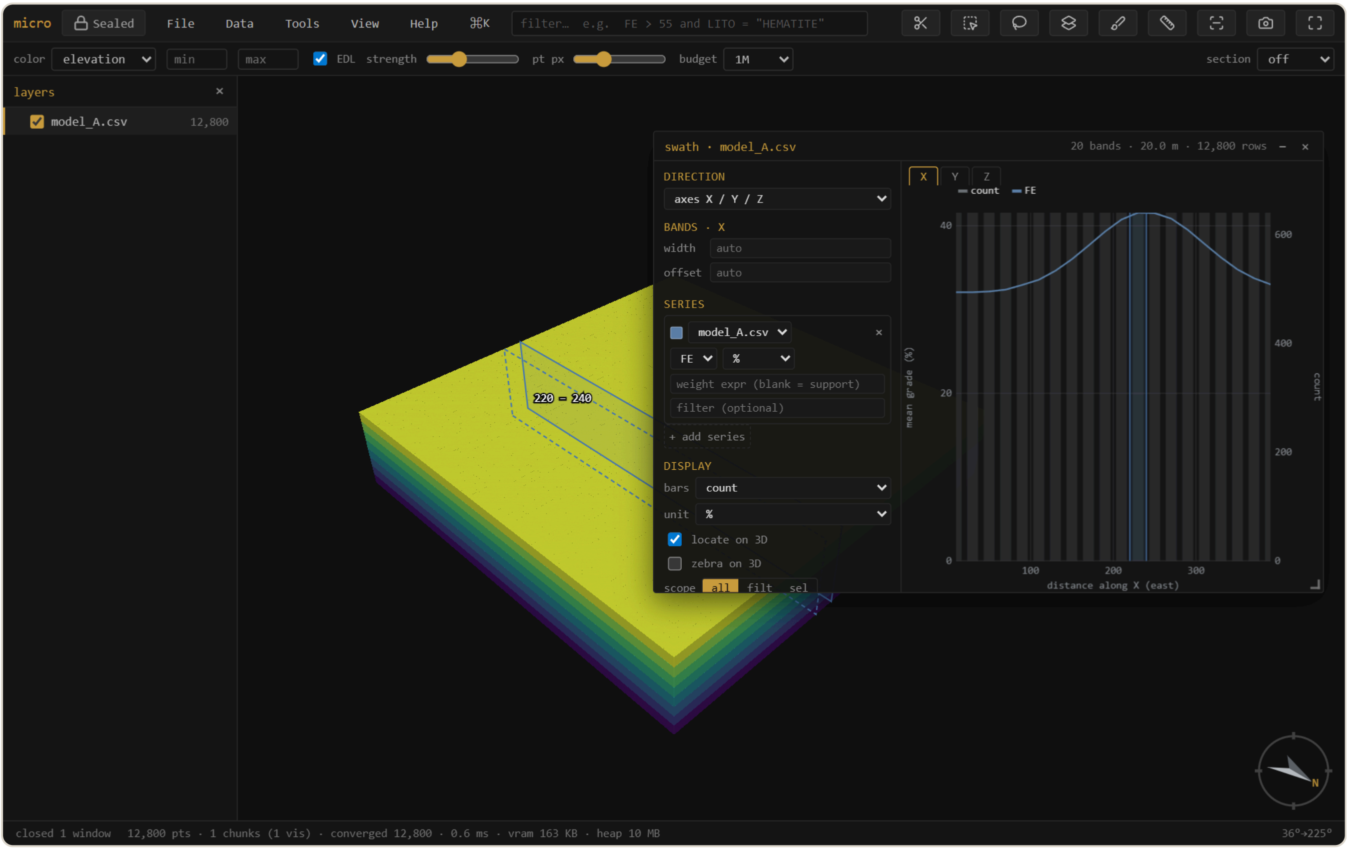
Zebra paints the alternate bands onto the 3D, so the slabs you see in space *are* the bins in the plot.



A swath along easting, with the bands zebra-striped onto the model. Because the plot and the stripes share one direction / width / offset, you read the profile against the deposit it came from.

The band ghost — hover the plot, see the slab

With **locate on 3D** on (it is by default), hovering the swath plot lights that band on the 3D as a translucent slab outline — so a dip in the profile points straight at the benches or the zone that caused it. **Drag** across the plot to hold a band *range* (violet), then turn it into a real 3D **selection** right from the menu that appears — and from there it's linked brushing, a 0/1 column, or an export like any other selection.

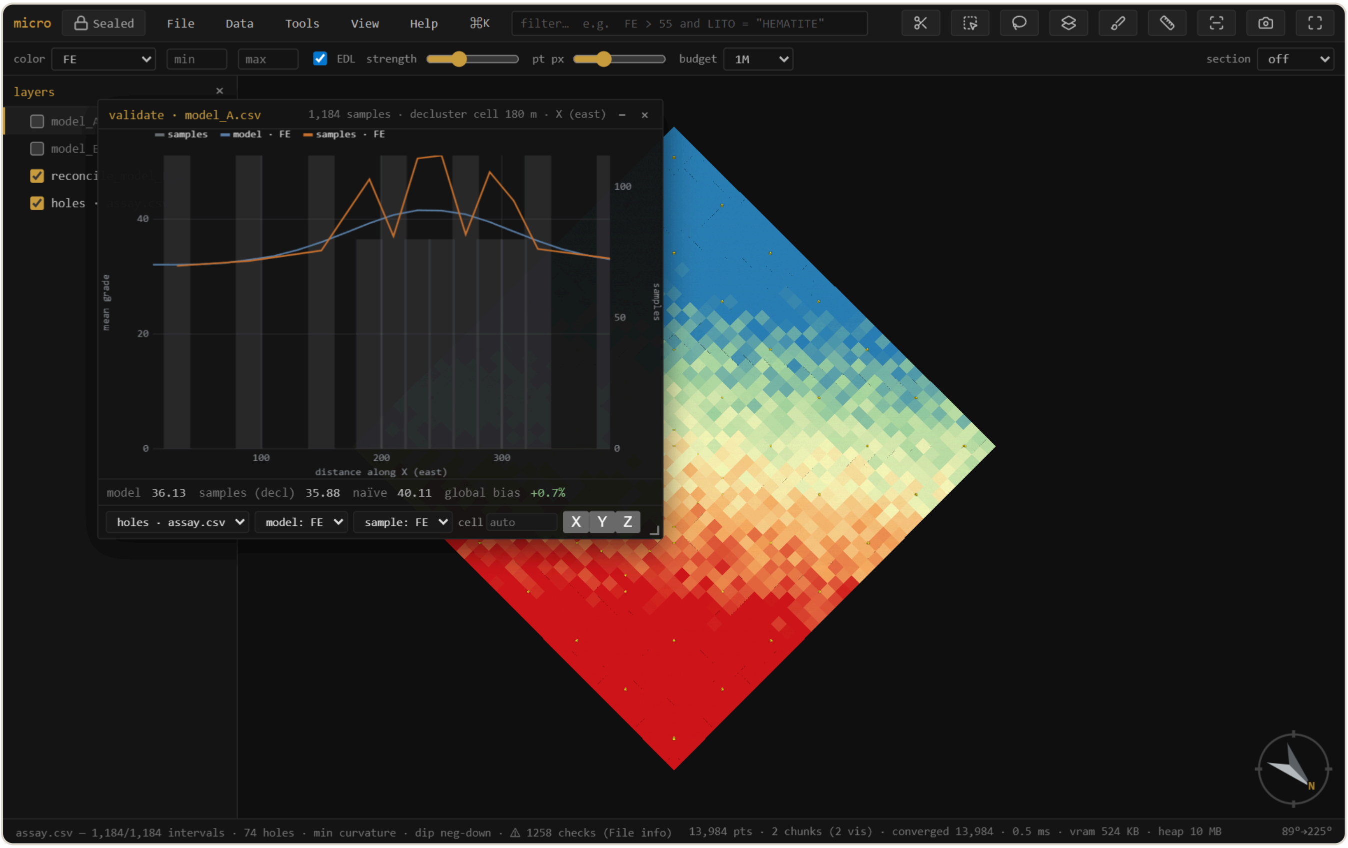


The band ghost: the bin under the cursor (shaded in the plot) is the slab outlined on the model. Drag a range and the menu offers *Select rows in…* — the plot becomes a selection tool.

8 Validate vs drillholes

Is the model consistent with the samples it was built from? **Validate vs drillholes…** takes a block model and a drillhole layer, **declusters** the sample grades (spatial sampling over-drills high grade, so a naïve sample mean is inflated), and reports the **global bias** (model mean vs declustered sample mean) plus a **model-vs-sample swath** along X/Y/Z.

> **Note —** The validation window is **temporarily hidden from the menus** while it's rebuilt on the new analysis chassis (per-series units, weight expressions, scopes). The machinery it uses — declustering, sample swaths — is unchanged; meanwhile the swath window already overlays model vs drillhole-sample series directly.



Validation. The naïve sample mean (40.1) is inflated by clustered infill drilling; **declustered** it's 35.9, essentially matching the model (36.1) — a **+0.7% global bias**. The swath compares model (blue) and sample (orange) grade along easting, slab by slab.

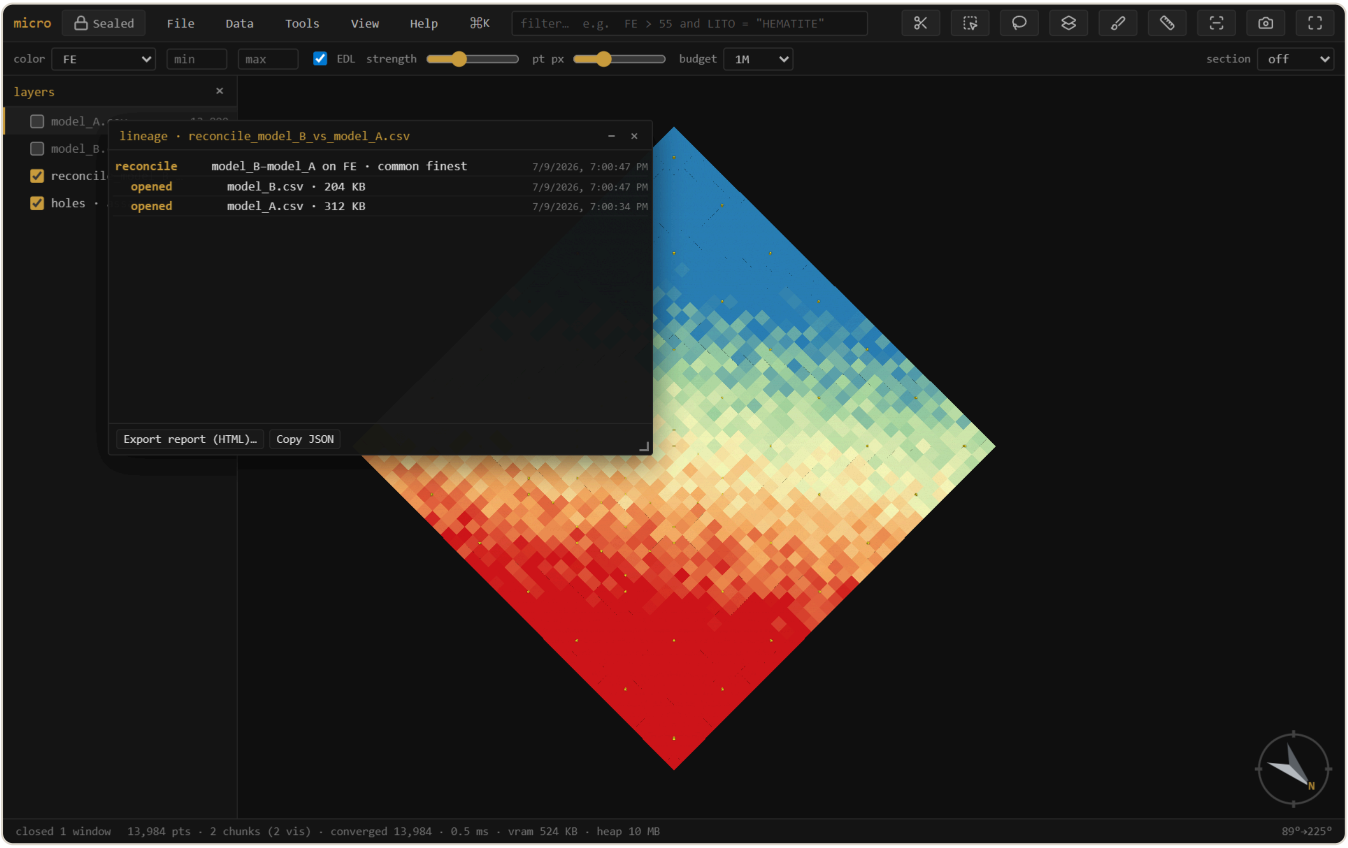
> **Note —** Samples are point support, blocks are block support, so a *global* grade offset between them can be real (change of support), not an error. The swaths are what flag *local*, conditional bias.

9 Linked brushing

The grade-tonnage and swath windows carry an **all / filt / sel** scope. **sel** follows the 3D selection: rubber-band a bench, a domain, or a suspect zone and the plot recomputes for just those blocks, live, as you brush. **filt** follows the **filter box** instead — type `LITO = "HEMATITE"` and every open analysis window scoped to it re-runs with each new expression. The analysis becomes interactive regional exploration, not a whole-model summary.

10 Lineage & provenance

Every derived layer remembers *how it was made*. Opening a file is a leaf; a join, reconcile, or optimize wraps its source layers with the operation, its parameters, the build, and a timestamp — a tree. **Lineage...** shows that tree and exports a readable HTML **report** you can attach to a sign-off. Parquet exports carry the lineage embedded in the file, so the provenance travels *inside* the data.



The provenance tree for a reconciliation: the result wraps the two opened models. Export it as an HTML report, or read it back from any Parquet micro wrote.

11 Grids

GeoTIFF / Surfer `.grd` / ESRI `.asc` open as heightfield layers (cells are pickable records; huge DEMs decimate for display while the full grid stays for evaluation). A mesh right-click → **Rasterize to grid…** renders it onto a DEM (the surface↔grid converter, with fill / extrapolation for gaps). **New…** → **grid / block model** creates empty grids from parameters, covering a layer's bounds, or — **cover compatible models** — on a lattice that covers and is compatible with several models at once (the reconcile target). Grid layers export as ESRI ASCII.

12 The Parquet project store

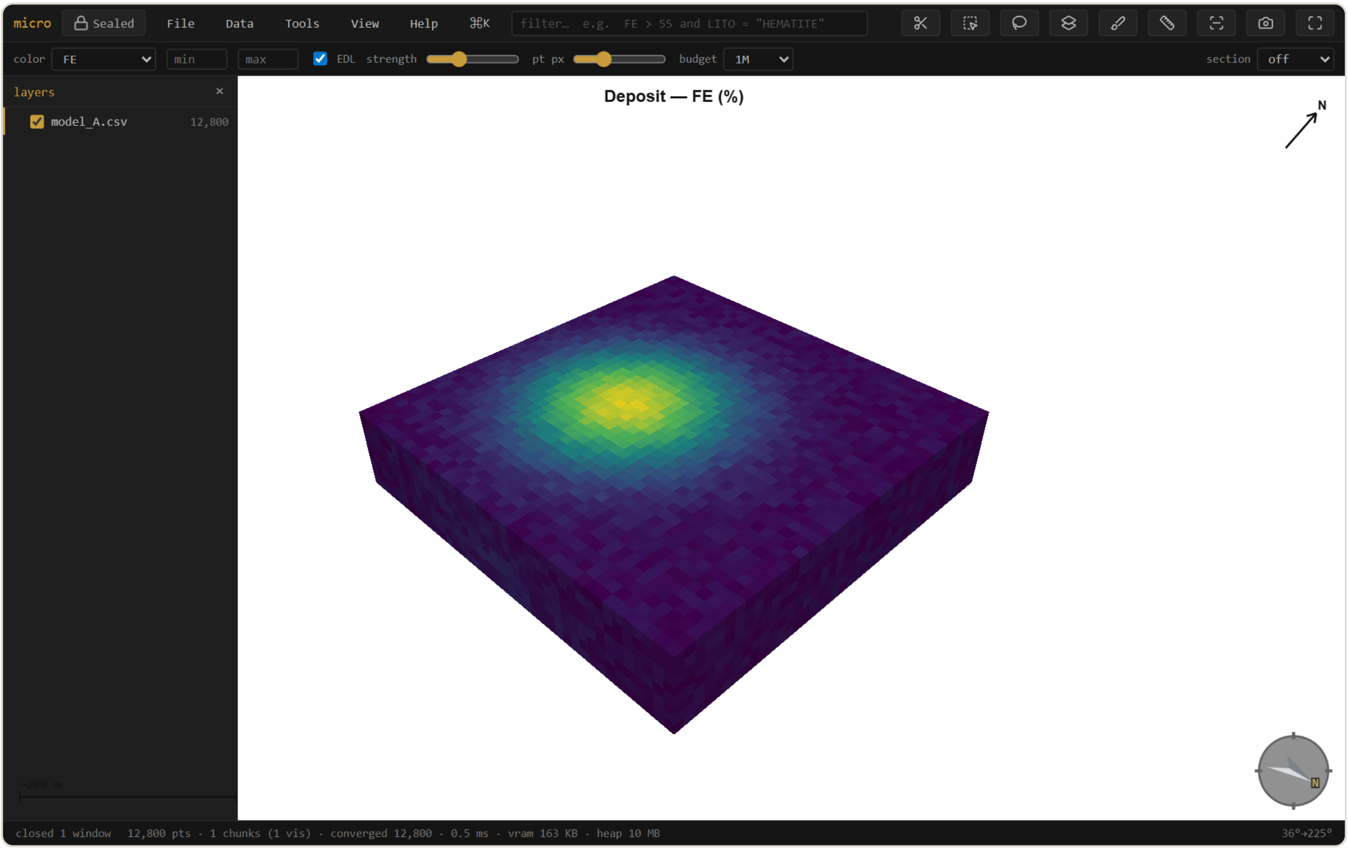
Store as Parquet… (right-click a CSV/ `.dm` model) converts it Morton-sorted, so each row-group's footer becomes a tight spatial index and queries skip whole row-groups they can't match. The dialog says exactly what will happen — per-layer rows, sizes, and a **keep original** option — and the same conversion runs **when you save a project** (a one-time, remembered choice: always / ask / off). Parquet is micro's project store. Painting stays editable through the reorder, the file self-describes (grid, extents, categories in its metadata, so it reopens with no discovery pass), and the whole thing round-trips. Parquet block models also open **streamed**, holding nothing decoded, with predicate and spatial push-down on the filter. Truly huge conversions (tens of millions of blocks) stream in file order rather than sorting in memory — the footers stay spatially tight because block-model files are written in grid order anyway.

Sidecars — the project remembers its scans

Files that live in a project folder get a small `.meta.json` **sidecar** written at save time: the discovery result (grid, extents, sub-blocking, categories — so reopening skips the sweep entirely, even on a 13 GB `.dm`), the column statistics (the styling histograms wake up pre-scanned), and a **band index** that lets filters on CSV and `.dm` files skip whole stretches of the file that provably can't match. Sidecars are a cache: delete one and micro just re-scans.

13 Figures & decorations

View → Decorations & figure… toggles a **scale bar**, **north arrow**, colour **legend**, and **title** onto the viewport — they show live and are captured verbatim by **Export figure** (at 1× / 2× / 3× scale for print). Legends are **per layer** — stack one per model when several share the view, each with its own title. Set a **background** (white or paper for print; the decorations flip to dark ink to stay readable). The exported PNG is what you see, ready for a report — and it pairs with the lineage report for a fully auditable figure. The analysis plots have their own ⤓ **png / ⧉ copy / ⧉ table** buttons.



A report-ready figure: white background, scale bar, north arrow, grade legend, and title — composited onto the export exactly as shown.

14 Projects & export

In a Chromium browser, **File → Open project folder** keeps your layers, styling, filters, selections, camera, section, saved views — and your open **analysis windows** (see Windows) — in a plain folder (`project.json` + the data files). `Ctrl+S` saves.

The folder is organized **by data kind** — `models/` `drillholes/` `clouds/` `meshes/` `grids/` — with each file's sidecar beside it. **Tidy project folder** migrates an older flat project in place, and **Scan folder for new data** treats the folder as an inbox: drop a file in by hand (or from another tool) and micro picks it up. A loose layer (opened from outside the folder) offers **Copy into project** from its context menu — always with a sized consent step, never silently.

Export rows… writes any layer back out — scope **all** / **filter matches** / **selection**, every column including calculated and painted ones, as CSV or Parquet. Opening a `.dm` and exporting *is* the `.dm` →CSV (or →Parquet) converter.

Recipes — save an analysis to run again

save as recipe… (next to Run in the grade–tonnage and swath windows) writes the current setup as a named YAML file in `recipes/` — every series, unit, weight expression, band, scope, and range. **Tools → Recipes** lists them; picking one opens the tool configured **and runs it**. Recipes are plain, commented, hand-editable files — copy one between projects, keep an office standard set, drop one into the folder and **Scan folder** picks it up. Series reference layers **by name**; if the named layer isn't loaded, one question asks which loaded model to use instead — that's the whole parameter system. A recipe is data, never code: the strict YAML subset can't carry a script.

```yaml
# micro recipe — grade-tonnage. Hand-edit freely; series reference layers by NAME
name: "Monthly GT"
tool: "gt"
params:
  nCut: 15
  gradeUnit: "pct"
  series:
    - layer: "model.parquet"
      col: "FE"
      weight: "SG"
```

Mesh export

Export mesh… (right-click a mesh layer; multi-selections and groups get **Export N meshes…**) writes **OBJ**, **PLY**, **MSH** (Leapfrog/ARANZ), or **LFM**. OBJ and LFM are multi-mesh — one named object per layer, with each layer's tint carried as the LFM colour — while PLY and MSH merge multiple layers into one mesh. Exports re-read the source geometry, so coordinates leave in **full double precision** (the PLY writer uses `double` — at UTM northings, float32 would cost half a metre). A Datamine wireframe pair in, an LFM out: micro is also a mesh-format converter.

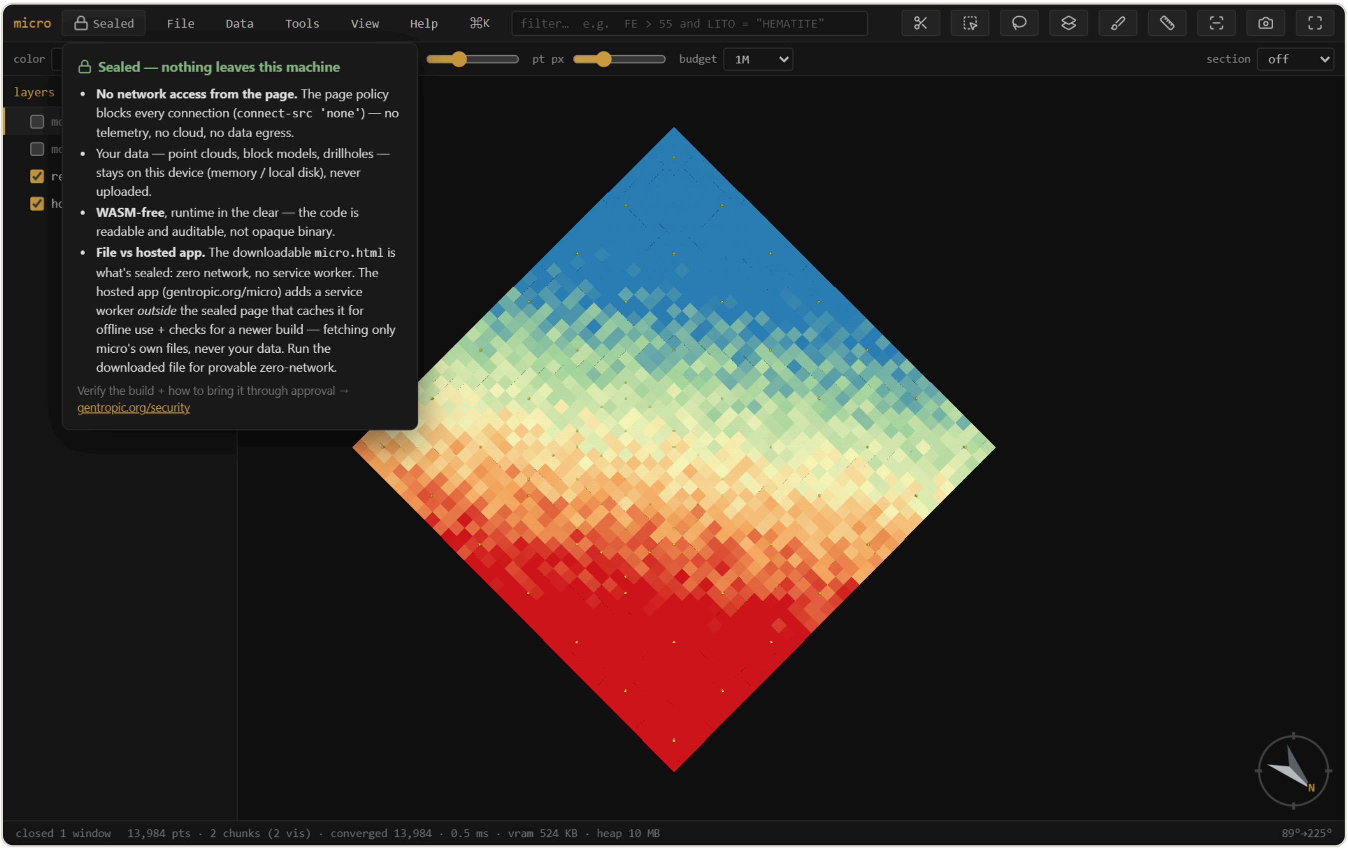
15 Windows

The analysis, table, and paint windows are all draggable and pinned to their layer (open several at once). **View → Windows…** lists every open window — click to focus, or **Cascade** and **Close all** to tidy up.

Grade–tonnage and swath setups **persist in the project**: every series, unit, weight expression, band width, scope, and manual range is snapshotted on each Run and on close, and a window left open when you save is **pinned** — reopening the project brings it back in place, configured, reading *restored — Run to compute*. Run stays manual, so a huge project never surprise-scans on open.

16 **Security: Sealed**

micro is **Sealed**: it makes **no network requests** — a browser security policy (`connect-src 'none'`) blocks every connection, so there is no telemetry, no cloud, and no auto-update. Your data — point clouds, block models, drillholes — stays on your device. The build is **WASM-free** with the runtime in the clear, so the code is readable and auditable rather than opaque binary.



The **Sealed** badge (top-left) explains the posture and links to the verifier. For confidential resource data, this is the point: nothing leaves the machine.

Verify the build and read how to bring it through approval at gentropic.org/security.

17 **Install & offline**

micro runs at gentropic.org/micro — install it as an app from the browser, or download `micro.html`: one file, no installer, that you can keep, e-mail, drop on a shared drive, or run fully offline by double-clicking. There is nothing to set up and nothing that phones home. Works in current Chrome / Edge / Firefox; project folders and mounts need a Chromium browser.

A Reference

Keyboard

Key	Action	Key	Action
`⌘/Ctrl K`	command palette	`/`	focus the filter
`f`	fit the data	`p`	layers panel
`n s e w`	look level that way	`d` / `u`	plan / from below
`o`	ortho / perspective	`x`	section on/off
`k`	knife — drag a section (⇧ snap 15°, ⇧⌥ 5°)	`l` / `⇧L`	face the section / its back
`,` `.`	step one section slab	`Ctrl wheel`	scrub the section
`m`	measure	`i`	file info
`r` / `v`	select rectangle / lasso	`b`	paint a column
`t`	attribute table	`[` `]`	step the active layer
`h` / `⇧H`	hide–show / solo the active layer	`F2`	rename the active layer
`⇧B`	block edges	`⇧P`	properties
`Ctrl S`	save project	`Ctrl Z`	undo paint
`Esc`	clear selection / cancel		

The filter language

Comparisons `> >= < <= = !=`, boolean `and` / `or` / `not`, `between a and b`, `in (a, b, c)`, over any column (text values in quotes). The same expressions define calculated columns. Autocomplete offers columns, functions, and category values — `Tab` / `Enter` accepts.

micro — part of the Auditable project, Geoscientific Chaos Union. Engine: `@gcu/condenser`; declustering: `@gcu/gsjs`. MIT.
Screenshots are generated from the app by an automated harness, so they track the current build. Docs for micro — see the in-app **Help** menu for the terse quick reference.